

Move семантики

A&& - именована реф-р

A& - именована реф-р

Типове стойности в C++:

- **lvalue** - израз, който има адрес в паметта

`int a = 7; (a - lvalue)`

- име на съществуваща пром-ва

- извикване на ф-я, която връща по ref $\rightarrow$ `int & f();`
`f() = 7;`

- **rvalue**

7, "AB", true

- литерали

- извикване на ф-я, която връща по копие

RVALUE

- **xvalue (expiring value)**

- обект към края на живота си

`f(int &a)` - lvalue

~~`f(3);`~~

`f(const int &a)` - lvalue и rvalue

`f(3);` ✓

(const използва името на rvalue-то)

Както и при копи конструктора, copy assignment оператора и деструктора, компилаторът автоматично ще синтезира move конструктор и move assignment оператор.

Единствената разлика тук е, че условията, при които той прави това са различни.

За разлика от копи операциите, за някои класове компилаторът изобщо не създава move конструктор и move assignment оператор.

В частност, ако един клас дефинира свой copy конструктор, copy assignment оператор или деструктор, то move конструкторът и copy assignment операторът няма да бъдат синтезирани. Ако един клас няма move операции, то се използват copy такива (даже върху временни обекти, като точно това целим да избегнем), ако такива има разбира се.

Компилаторът ще създаде move конструктор и move assignment оператор ако класът няма дефиниран copy control и ако всеки член може да бъде "преместен".

Компилаторът може да премества вградени типове, а също и класове, които имат съответната move операция дефинирана. При примитивните типове данни местенето е просто копиране.

Ако експлицитно помолим компилатора да генерира move операция чрез `=default`, и компилаторът не успее да синтезира такъв, то той бива маркиран като `=deleted`.

! Какво е rvalue reference ?

88

$f(\text{int } \&a)$ - приема само rvalue

$\Rightarrow f(3) \checkmark$

$f(n) \times$

→ ОЧАКВА ПОДАДЕНАТА ПРОМ/КОНСТАНТА ДА НЕ СЕ ПОЛЗВА СЛЕД Ф-ТА

`addStr(string &str)`

↳ ПЛУК МОЩЕ ДА ОТКРАЖЕМ
ДАНИТЕ НА `str`, А НЕ ДА ГИ КОПИРАМЕ

Move constructor

- приема rvalue & респ.

- създава нов обект, "крадейки" данните от подаления
като параметър (и го оставя в състояние, в което данните не са
налични при изтичането си)

- при примитивните типове е просто копиране.

- ПРЕОБРАЗУВА lvalue в rvalue (xvalue)

- КАЗВАМЕ ЧЕ ОТ ПОДАДЕНО LVALUE МОЩЕМ ДА КРАДЕМ,
ЗАЩОТО НЕМА ДА СЕ ПОЛЗВА СЛЕД Ф-ТА

STD::MOVE

Вашен пример: ⚡

```
int main() {
```

```
    A obj; // AC
```

```
    f(std::move(obj)); // няма деструктор, защото е xvalue
```

```
    f(A(obj)); // AC и ~AC
```

при $f(\text{int } \&a)$
 $f(\text{std::move}(a))$

```
int main() {
```

```
String s1 = "AB"; // k-p
```

```
String s2(s1); // конч k-p
```

```
String s2(std::move(s1)) // move k-p
```

```
String s3; // jedp k-p
```

```
s3 = std::move(s2); // нуго
```

```
} ~String()
```

```
f(string str) {  
}
```

```
f(string("ABC")) // k-p + g-p
```

```
f(s1) // конч k-p + g-p
```

```
f(std::move(s2)) // move k-p + g-p
```

```
g(A&&) {  
}
```

```
{ A obj; A()
```

```
g(std::move(obj));
```

```
} ~A();
```

(, нуго не
се вика мук ☹

Комп. автоматично създава move семантика, ако не сме разписали kк и kвиz и kип. може да бъде "преместен". Ако нямате move се използва copy оп.
Ако пополим комп. да създаде move операция през default и той не успее, тогава тя се извършва като delete.

ИЗКЛЮЧЕНИЯ

ИЗКЛЮЧЕНИЕ -
СИГНАЛ, ЧЕ СЕ Е СЛУЧИЛО
НЕЩО НЕРЕДНО

```
throw <обект>; - спира изп-то в ср-та  
и търси има ли кой  
да обработи грешката,  
ако:
```

не => std::terminate

да => stack unwinding

```
catch (int a) {  
    // обработка  
}
```

```
main() {
```

```
    try {
```

```
        f();
```

```
    }  
    catch (int a) {  
        cout << a;
```

```
    }  
    catch (string &s) {  
    }  
}
```

```
f() {
```

```
    A obj;
```

```
    g();
```

```
}
```

```
g() {
```

```
    X obj;
```

```
    throw "AB";
```

```
    Y obj2;
```

```
}
```

NA()

X()

Гледа кой 1-ви
ще мине.

⊙ Провокура прагмат
на копие => const A&

```
try {  
}  
catch (int a) {  
}  
catch (double d) {  
}  
catch (A obj) {  
}  
catch (...) {  
}
```

ХВАЩА
ВСИЧКО

Кога да хвърляме изключения?

- грешки входни данни
- изклю-та НЕ трябва да стигнат до потребителя
- отговорността е наша да ги обработим, ако ползваме класа

assert vs. exception

↓
за наши
грешки
(системни)

↓
за грешки с
входни данни

има ф-я
what()

(може да
взвучи съобщ-то
на грешката)

`std::bad_cast`
(грешка при
кастване)

Видове грешки

`std::runtime_error`

`std::exception`

`std::bad_alloc`

(грешка при
заделяне на памет)

`std::logic_error`

(нарушаване на инвариант-те
на класа)

⚠ При изк-я в кон-те се изчислява да няма
незамърсени външни ресурси.

Af

```
char *str1;  
char *str2;  
}
```

A()

// защото ~A() не е бил извикан //

```
str1 = new char[20];
```

```
try {
```

```
str2 = new char[20];
```

```
}
```

```
catch (const std::bad_alloc & e)
```

```
{  
    delete[] str1;  
}
```